ИТМО

Алгоритмы и структуры данных

Лекция 5

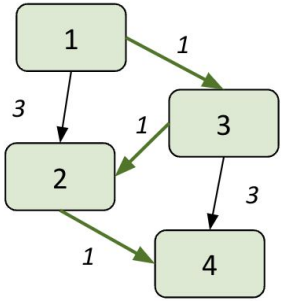
Структуры данных для непересекающихся множеств

Начнем с вопроса «Зачем»

Алгоритм Краскала (c++)

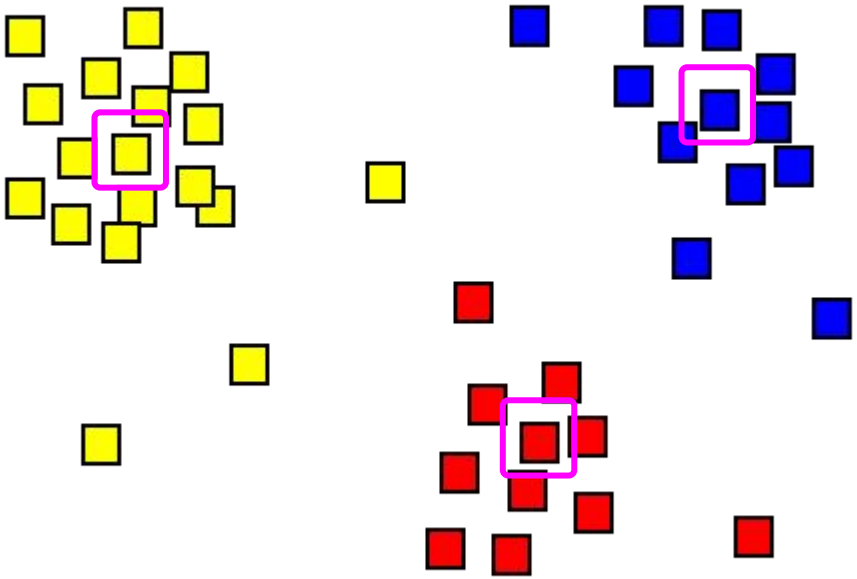
```
struct Edge {
   int from, to;
   int weight;  // Вес ребра

   bool operator<(const Edge &other) const {
       return weight < other.weight;
   }
};
```

```
class DisjointSet {
public:
   DisjointSet(const std::vector<Edge> &edges) {...}

   // Метод для нахождения корня множества (сжатие пути)
   int findRoot(int u) {...}

   void unionSets(int u, int v) {...}  // Метод для объединения двух множеств

private:
   std::unordered_map<int, int> parent; // словарь для хранения родителей
   std::vector<int> rank;   // Вектор для хранения рангов
};
```



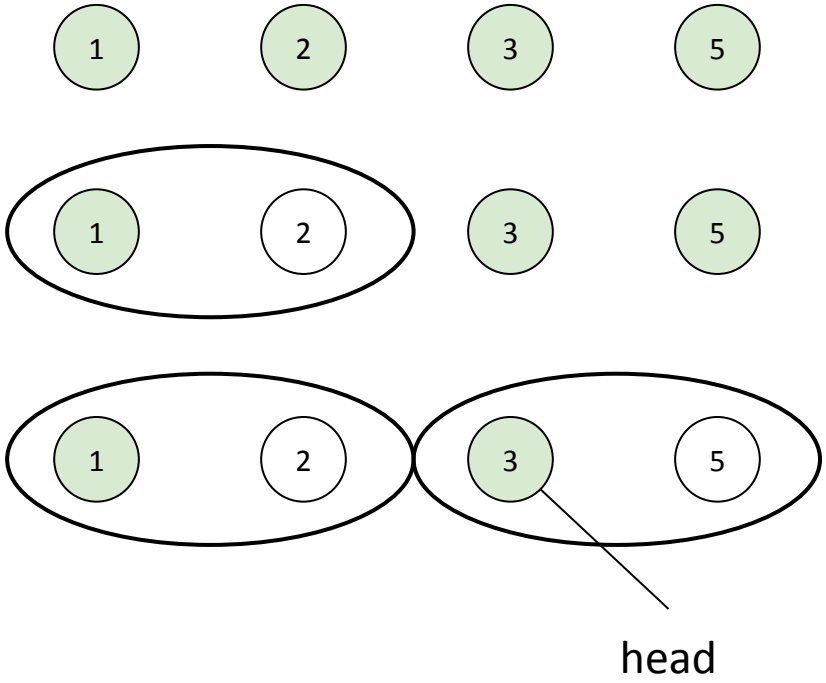
Начнем с вопроса «Зачем» 2

Кластеризация - рассматриваем каждый объект как множество



На каждом шаге определяем наличия объекта в кластере, и сливаем кластера

Disjoint Set Union



Свойства:

- Каждый элемент принадлежит только одному множеству
- Множества не пересекаются

Операции:

- ADD. Добавить в множество новое подмножество
- FIND. Поиск элемента в множестве
- UNION. Объединение множеств

Простая реализация через словарь

ИТМО

Пример – *{{**1**, 5}, {6}, {**8**, 9}}*

Используем словарь

Индекс	1	5	6	8	9
Ссылка	1	1	6	8	8

```
# Найти подмножество по элементу
def find(num: int) -> int | None:
    # Возвращаем head подмножества

    if num not in DSU: return None

    return DSU[num]
```

UNION: Нужно вот тут изменить ссылку на первый кластер

```
def union(a: int, b: int):
    a, b = find(a), find(b)
    for k, v in DSU.items():
        if v == b:
            DSU[k] = a
```

Какая сложность?

Простая реализация через словарь

ИТМО

Пример – *{{**1**, 5}, {6}, {**8**, 9}}*

Используем словарь

Индекс	1	5	6	8	9
Ссылка	1	1	6	8	8

UNION: Нужно вот тут изменить ссылку на первый кластер

```
# Найти подмножество по элементу
def find(num: int) -> int | None:
    # Возвращаем head подмножества

    if num not in DSU: return None

    return DSU[num]
```

O(1)

```
def union(a: int, b: int):
    a, b = find(a), find(b)
    for k, v in DSU.items():
        if v == b:
            DSU[k] = a
```

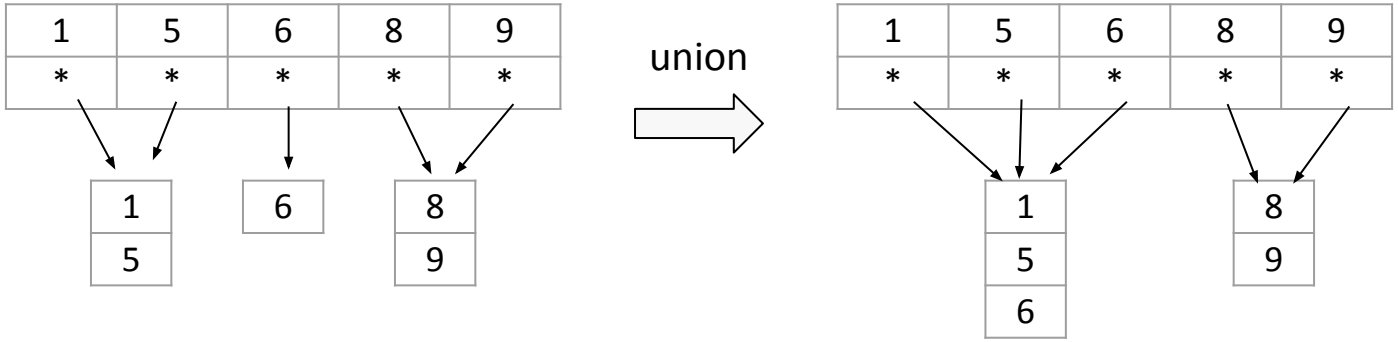
O(N)

Реализация через связный список

ИТМО

Пример - {{**1**, 5}, {6}, {**8**, 9}}

Используем словарь + связный список




```python
def find(num: int) -> int | None:
    # Возвращаем head подмножества

    if num not in DSU: return None

    return DSU[num]
```

```python
def union(a: int, b: int):
    a, b = find(a), find(b)  # LinkedList
    for value in b:
        a.add(value)
        DSU[value] = a
```

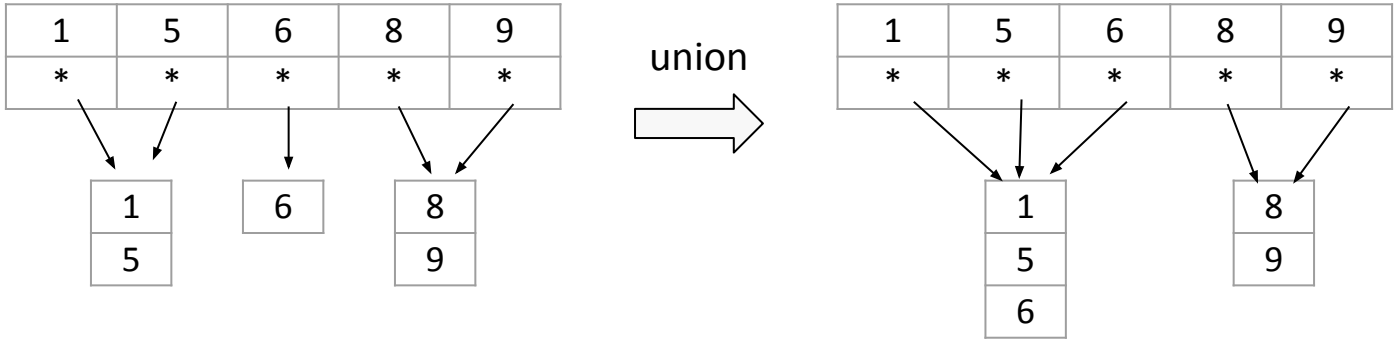
Какая сложность?

Реализация через связный список

ИТМО

Пример - {{**1**, 5}, {6}, {**8**, 9}}

Используем словарь + связный список




```python
def find(num: int) -> int | None:
    # Возвращаем head подмножества

    if num not in DSU: return None

    return DSU[num]
```

O(1)

```python
def union(a: int, b: int):
    a, b = find(a), find(b)  # LinkedList
    for value in b:
        a.add(value)
        DSU[value] = a
```

O(N)

На подумать

ИТМО

	Поиск	Объединение
Реализация на словаре	O(1)	O(N)
Реализация на связном списке	O(1)	O(N)

Стало лучше или хуже?

Улучшение реализации на связном списке

ИТМО

Было

```
def union(a: int, b: int):
    a, b = find(a), find(b)
    for value in b:
        a.add(value)
        DSU[value] = a
```

Стало

```
def union(a: int, b: int):
    a, b = find(a), find(b)
    a, b = (a, b) if len(b) < len(a) else (b, a)
    for value in b:  # итерируем меньший список
        a.add(value)
        DSU[value] = a
```

Улучшение реализации на связном списке

ИТМО

Было

```
def union(a: int, b: int):
    a, b = find(a), find(b)
    for value in b:
        a.add(value)
        DSU[value] = a
```

Стало

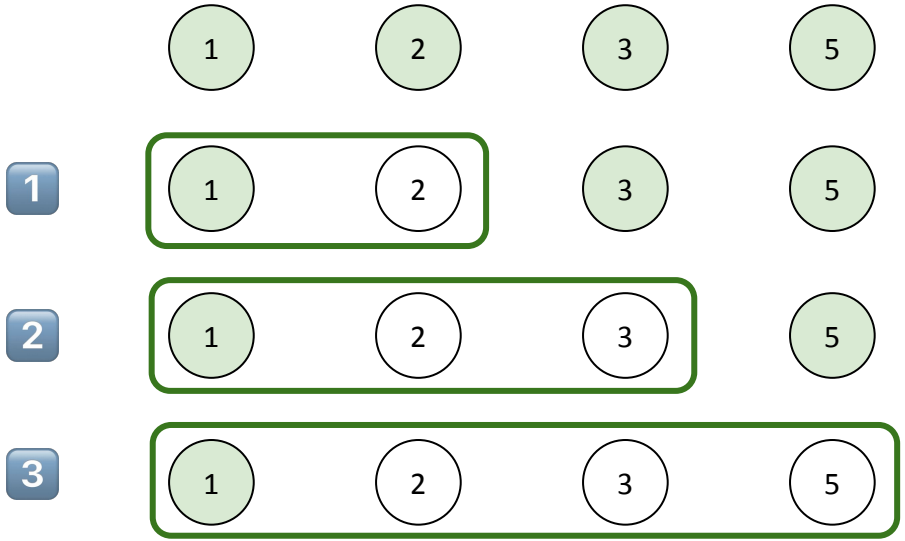
```
def union(a: int, b: int):
    a, b = find(a), find(b)
    a, b = (a, b) if len(b) < len(a) else (b, a)
    for value in b:  # итерируем меньший список
        a.add(value)
        DSU[value] = a
```

Оценим разницу сложности

Оценим разницу сложности (лучший случай)

ИТМО

Мержим все подмножества пока не останется только ОДНО подмножество



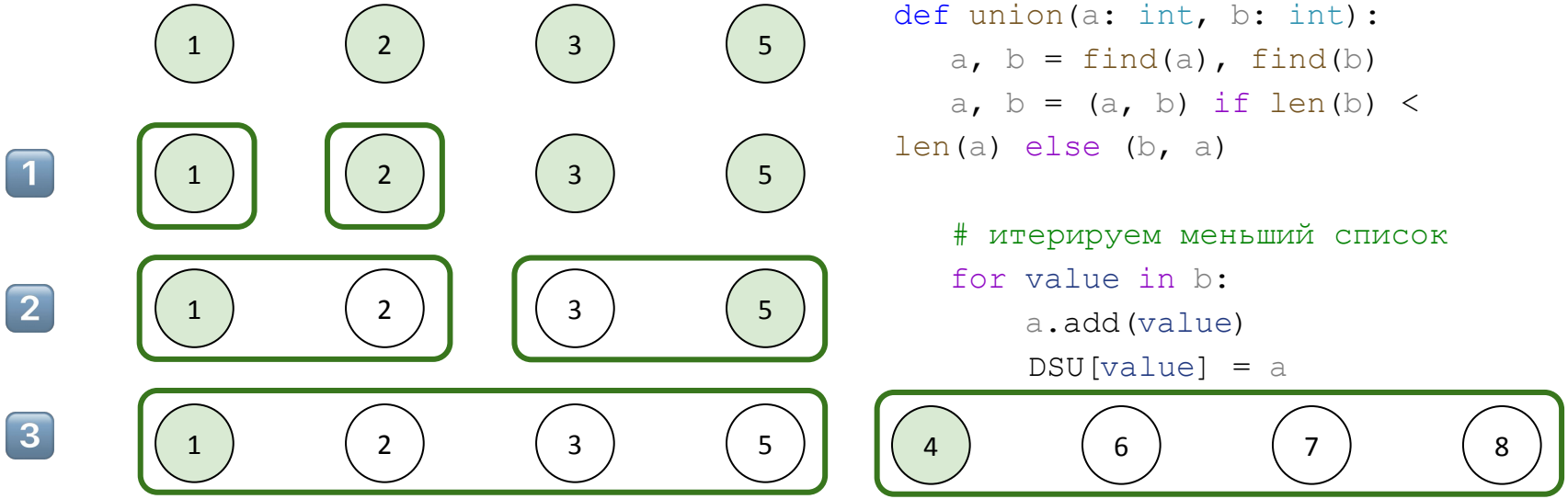

```
def union(a: int, b: int):
    a, b = find(a), find(b)
    a, b = (a, b) if len(b) <
len(a) else (b, a)

    # итерируем меньший список
    for value in b:
        a.add(value)
        DSU[value] = a
```

Всего N - 1 операций

Оценим разницу сложности (худший случай)

ИТМО

Мержим все подмножества пока не останется только ОДНО подмножество




```
def union(a: int, b: int):
    a, b = find(a), find(b)
    a, b = (a, b) if len(b) <
len(a) else (b, a)

    # итерируем меньший список
    for value in b:
        a.add(value)
        DSU[value] = a
```

Всего N*logN операций

O*(logN)

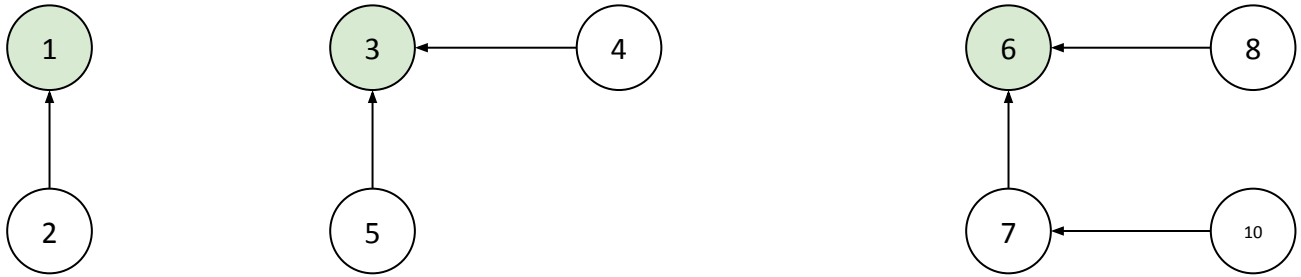
Получается

ИТМО

	Поиск	Объединение
Реализация на словаре	O(1)	O(N)
Реализация на связном списке	O(1)	O*(logN)

Что дальше?

Реализация через деревья

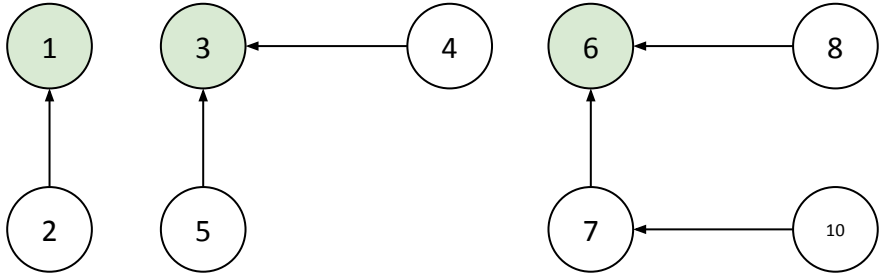


Но с хитрой структурой

```
DSU_tree: dict[int, Node]  = {
    1: Node(1),
    5: Node(5),
    6: Node(6),
    8: Node(8),
    9: Node(9),
}
```

Реализация через деревья 2

ИТМО




```
DSU_tree: dict[int, Node]  = {
    1: Node(1),
    5: Node(5),
    6: Node(6),
    8: Node(8),
    9: Node(9),
}
```

```
def find_tree(num: int) -> Node | None:
    if num not in DSU_tree: return None

    node = DSU_tree[num]
    while node.val_ != node.parent_.val_:
        node = node.parent_

    return node
```

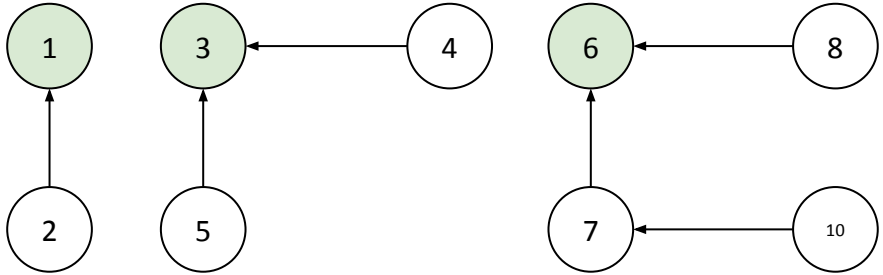
O(?)

```
def union_tree(a: int, b: int):
    a, b = find_tree(a), find_tree(b)  # Node
    # a >= b
    a, b = (a, b) if len(b) <= len(a) else (b, a)

    b.parent_ = a
    a.size_ = a.size_ + b.size_
```

O(?)

Реализация через деревья 2

ИТМО




```
DSU_tree: dict[int, Node]  = {
    1: Node(1),
    5: Node(5),
    6: Node(6),
    8: Node(8),
    9: Node(9),
}
```

```
def find_tree(num: int) -> Node | None:
    if num not in DSU_tree: return None

    node = DSU_tree[num]
    while node.val_ != node.parent_.val_:
        node = node.parent_

    return node
```

O(logN)

```
def union_tree(a: int, b: int):
    a, b = find_tree(a), find_tree(b)  # Node
    # a >= b
    a, b = (a, b) if len(b) <= len(a) else (b, a)

    b.parent_ = a
    a.size_ = a.size_ + b.size_
```

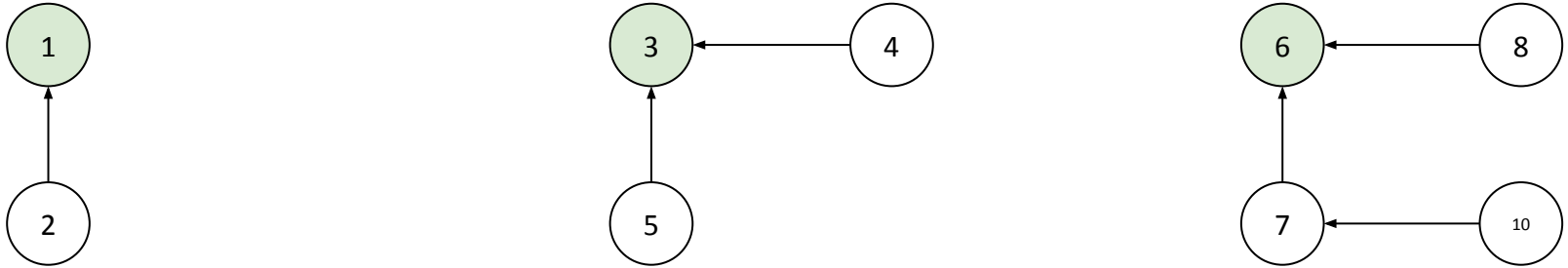
O(logN)

Текущий статус

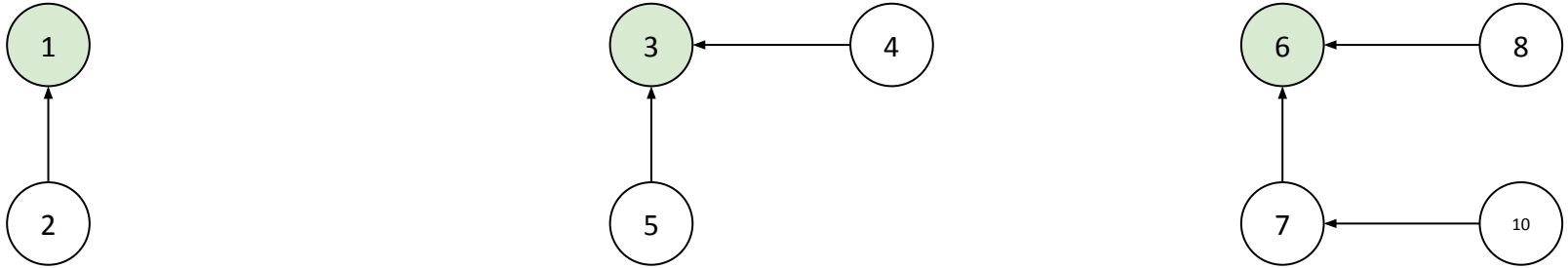
ИТМО

Реализация	Поиск	Объединение
На словаре	O(1)	O(N)
На связном списке	O(1)	O(N)
На связном списке (модифицированная)	O(1)	O*(logN)
На деревьях	O(logN)	O(logN)

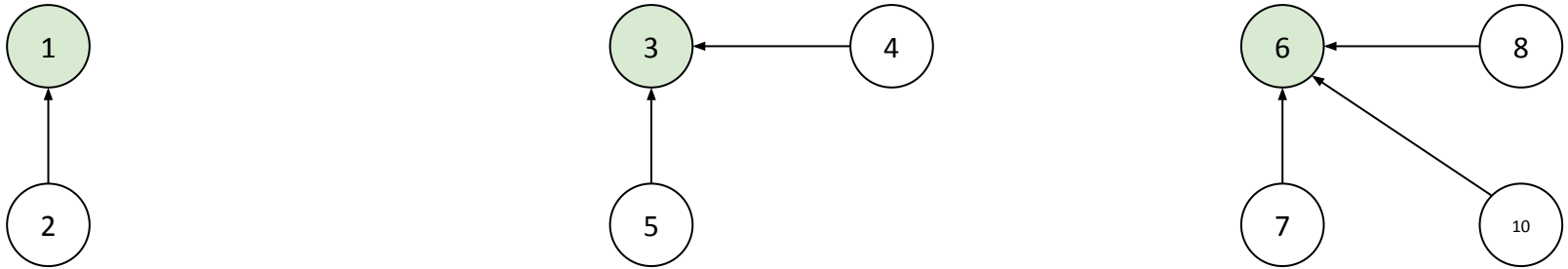
Можно еще немного улучшить



Можно еще немного улучшить



При операции find меняем ссылку в детях на корень



Текущий статус

ИТМО

Реализация	Поиск	Объединение
На словаре	O(1)	O(N)
На связном списке	O(1)	O(N)
На связном списке (модифицированный)	O(1)	O*(logN)
На деревьях	O(logN)	O(logN)
На деревьях (модифицированный)	O(log*N)	O(log*N)

где O(log*N) - сколько раз нужно применить логарифм, чтобы получилось меньше 1 (вроде)

$10^{80} \rightarrow \log(10^{80}) = 80 \rightarrow \log(80) = 1.9 \rightarrow 0.2$ (всего 3 операции)

Мне кажется

Или у вас есть вопросы?

ITsMOre than a UNIVERSITY

Ivan.Khodnenko@itmo.ru